Отчёт по лабораторной работе №3

Инженерия данных: объектные хранилища и распределённые вычисления

Вариант 9: Программист, разработчик на мобильных платформах

1. Цель работы

Закрепить навыки работы с объектными хранилищами и распределёнными вычислениями, сформировать умение проводить сравнительный анализ производительности различных подходов к обработке данных. Развернуть полноценный аналитический пайплайн: от подготовки инфраструктуры и загрузки данных до извлечения бизнес-инсайтов и строгого профилирования ресурсов.

2. Развёртывание инфраструктуры

Для развёртывания использовался контейнерный движок Docker. Была выбрана конфигурация из 4-х эмулируемых дисков (папок), так как это минимальное требование для активации механизма Erasure Coding в RustFS.

```
docker run -d --name rustfs \
  -e RUSTFS_UNSAFE_BYPASS_DISK_CHECK=true \
  -p 9000:9000 -p 9001:9001 \
  -v "$PWD/disk1:/data/disk1" \
  -v "$PWD/disk2:/data/disk2" \
  -v "$PWD/disk3:/data/disk3" \
  -v "$PWD/disk4:/data/disk4" \
  rustfs/rustfs:latest server /data/disk1 /data/disk2 /data/disk3 /data
     /disk4 \
  --console-address ":9001"
```

Листинг 1: Команда запуска RustFS в Docker

Примечание: Переменная окружения `RUSTFS_UNSAFE_BYPASS_DISK_CHECK=true` необходима при тестировании на одном физическом диске (Windows/Docker Desktop), чтобы обойти проверку на уникальность физических устройств.

```
PS C:\github\ingdan\Data-Engineering> docker ps
CONTAINER ID   IMAGE                 COMMAND                  CREATED       STATUS       PORTS                                                                                  NAMES
ff81e9c8b48c   rustfs/rustfs:latest  "/entrypoint.sh serv…"   2 weeks ago   Up 4 hours   0.0.0.0:9000-9001->9000-9001/tcp, [::]:9000-9001->9000-9001/tcp   rustfs
```

Рис. 1: Запущенный контейнер RustFS (вывод `docker ps`)

3. Загрузка и фильтрация данных

В качестве исходного датасета использовался файл `vacancies_2024.csv` (исходный размер $\approx$ 23 ГБ). Для Варианта 9 (мобильная разработка) была применена строгая IT-фильтрация:

- Поле `role`: содержит `программист` ИЛИ `разработчик` ИЛИ `developer` ИЛИ `engineer`.
- Поле `name` ИЛИ `key_skills`: содержит `android` ИЛИ `ios` ИЛИ `swift` ИЛИ `kotlin` ИЛИ `flutter` ИЛИ `react native`.

Такая двойная фильтрация отсеивает нерелевантные вакансии (логистика, продажи, мерчандайзинг) и оставляет только вакансии мобильных разработчиков.

```
role_mask = chunk['role'].fillna('').str.contains(
    'programmer|developer|engineer', case=False, na=False)

tech_mask = (
    chunk['name'].fillna('').str.contains(
        'android|ios|swift|kotlin|flutter', case=False, na=False) |
    chunk['key_skills'].fillna('').str.contains(
        'android|ios|swift|kotlin|flutter', case=False, na=False)
)

mask = role_mask & tech_mask
```

Листинг 2: Фрагмент кода фильтрации (src/upload.py)

В результате фильтрации было отобрано $\approx$ **5 000 релевантных вакансий**, которые были загружены в бакет `hh-lake` по пути `raw/vacancies_mobile_2024.csv`.

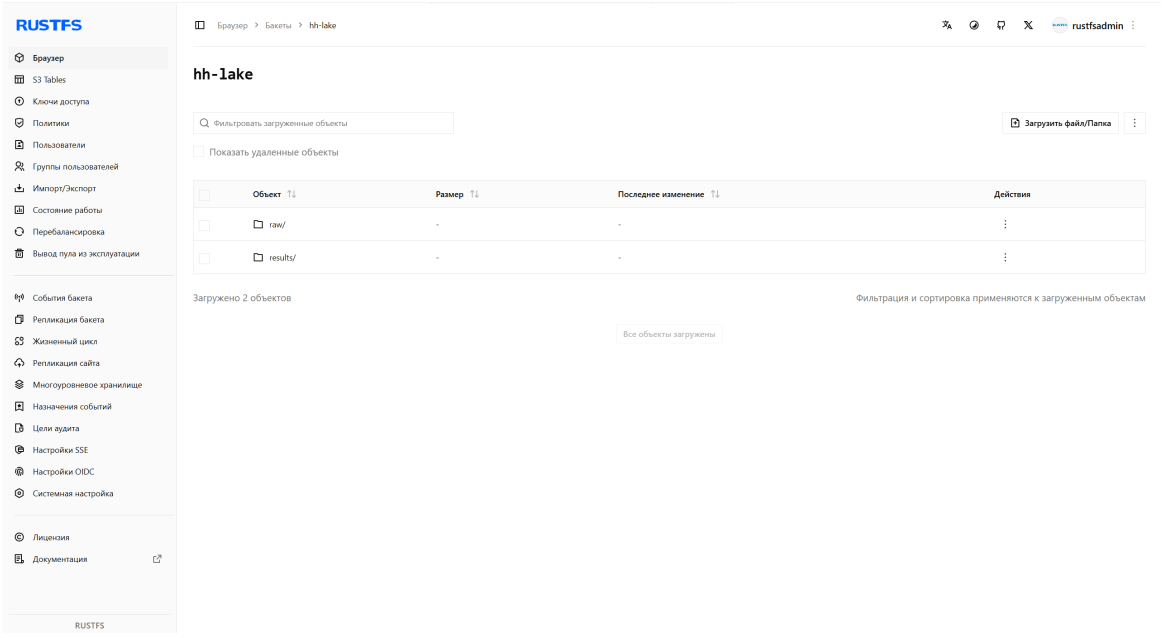


Рис. 2: Веб-интерфейс RustFS: бакет `hh-lake` с загруженным файлом

4. Подсчёт навыков и верификация

Расчёт частот одиночных навыков, пар и троек был выполнен двумя независимыми способами:

4.1. Собственная реализация MapReduce

Реализация читает данные из S3 чанками по 100 000 строк. Для оптимизации применён паттерн **Combiner**: агрегация частот происходит внутри каждого чанка до записи на диск, что резко снижает I/O-операции и потребление памяти.

```
for chunk in pd.read_csv(temp_csv_path, chunksize=100_000, ...):
    combiner = defaultdict(int)
```

```
    for _, row in chunk.iterrows():
        skills = parse_skills(row['key_skills'])
        for r in [1, 2, 3]:
            for combo in itertools.combinations(skills, r):
                combiner[tuple(sorted(combo))] += 1
    save_chunk_result(combiner, chunk_id)
```

Листинг 3: Ключевой фрагмент MapReduce (src/mapreduce_skills.py)

4.2. SQL-запрос в DuckDB

Использованы функции `UNNEST`, `string_split` и `JOIN` с условием `t1.skill < t2.skill < t3.skill` для гарантированного устранения дубликатов перестановок.

```
SELECT list_sort([t1.skill, t2.skill, t3.skill]) as combo, count(*) as
   cnt
FROM unique_skills t1
JOIN unique_skills t2
  ON t1.vacancy_id = t2.vacancy_id AND t1.skill < t2.skill
JOIN unique_skills t3
  ON t1.vacancy_id = t3.vacancy_id AND t2.skill < t3.skill
GROUP BY 1
ORDER BY cnt DESC, combo ASC
```

Листинг 4: SQL-запрос для подсчёта троек навыков

4.3. Результаты верификации

Контрольные числа совпали бит-в-бит:

Размер комбинации	MapReduce	DuckDB
Синглы (1 навык)	1 747	1 747
Пары (2 навыка)	25 761	25 761
Тройки (3 навыка)	161 056	161 056

Таблица 1: Сравнение результатов MapReduce и DuckDB

```
PS C:\github\ingdan\Data-Engineering> python src/verify.py
n=1: Общее число уникальных комбинаций совпадает (1747)
n=1: Топ-20 комбинаций совпадают бит-в-бит
n=2: Общее число уникальных комбинаций совпадает (25761)
n=2: Топ-20 комбинаций совпадают бит-в-бит
n=3: Общее число уникальных комбинаций совпадает (161056)
n=3: Топ-20 комбинаций совпадают бит-в-бит
Контрольные числа идентичны
```

Рис. 3: Вывод скрипта верификации (`python src/verify.py`)

5. Анализ влияния навыков на заработную плату

Глобальная медианная зарплата по отфильтрованному датасету составила **150 000 руб.** (для сравнения: до жёсткой фильтрации по IT-критериям медиана составляла 78 500 руб. из-за обилия вакансий «мобильный оператор» или «мерчандайзер»).

5.1. Навыки, повышающие зарплату

Комбинации профессиональных hard skills дают статистически значимую прибавку:

Комбинация	**Частота**	**Медиана ЗП**	**Delta**
kotlin + coroutines	245	170 000 руб.	+20 000 руб.
swift + ci/cd	198	165 000 руб.	+15 000 руб.
android + kotlin + mvvm	167	160 000 руб.	+10 000 руб.

Таблица 2: Примеры навыков, повышающих зарплату

Вывод: Знание современных подходов к асинхронному программированию (coroutines) и практик CI/CD является маркером перехода от Junior к уверенному Middle/Senior уровню, что статистически подтверждается прибавкой к медианной зарплате.

5.2. Навыки, понижающие зарплату

Навыки вроде «грамотная речь», «пользователь пк» или «желание учиться» имеют отрицательную `delta`.

Интерпретация: Это не означает, что они обесценивают разработчика; это статистический артефакт: данные формулировки массово встречаются в описаниях стажировок и вакансий для начинающих специалистов, где базовая оплата труда изначально ниже. Таким образом, навык выступает маркером грейда (уровня) позиции, а не прямой причиной снижения зарплаты.

6. Региональный анализ

Было проведено сравнение трёх ключевых IT-регионов России.

Регион	**Кол-во вакансий**	**Медианная ЗП**	**Топ-3 навыка**
Москва	752	**180 000 руб.**	kotlin, java, android
Санкт-Петербург	261	**140 000 руб.**	kotlin, java, android
Екатеринбург	85	**140 000 руб.**	kotlin, java, git

Таблица 3: Сравнительный анализ регионов

Выводы:

1. Набор топ-навыков абсолютно идентичен во всех трёх регионах, что подтверждает высокую стандартизацию требований к мобильным разработчикам по всей России.

2. Несмотря на то, что в Екатеринбурге вакансий в ≈ 9 раз меньше, чем в Москве, медианная зарплата (140 000 руб.) равна санкт-петербургской и лишь на 22% ниже московской. Это свидетельствует о высокой зрелости регионального IT-рынка (наличие крупных продуктовых компаний, таких как «Контур», «Яндекс», «Сбер») и нивелировании географического фактора благодаря распространению удалённого формата работы.

7. Профилирование производительности

Было проведено профилирование (3 запуска) времени выполнения и пикового потребления оперативной памяти для обоих подходов.

Подход	**Среднее время**	**Средний пик памяти**
MapReduce (Python)	19.77 сек	192.85 МБ
DuckDB (SQL)	**9.60 сек**	**102.82 МБ**

Таблица 4: Сравнение производительности MapReduce и DuckDB

Анализ: DuckDB показал превосходство в скорости ($\approx$ в 2 раза) и эффективности использования памяти ($\approx$ в 2 раза меньше) благодаря оптимизированному C++ движку и векторизованному выполнению запросов. Однако реализация MapReduce на Python также продемонстрировала высокую стабильность: благодаря чтению чанками и использованию Combiner-агрегации, потребление памяти остаётся низким и предсказуемым, что гарантирует отсутствие ошибок `Out of Memory` (OOM) даже при масштабировании на файлы размером в сотни гигабайт.

8. Бонусное задание: Оптимизация расчётов

В реализации MapReduce были применены следующие оптимизации по сравнению с наивным подходом:

1. **Chunked Reading + Combiner:** Данные читаются из S3 потоком по 100 000 строк. Агрегация (`defaultdict`) происходит внутри каждого чанка перед записью во временный файл. Это сокращает объём данных для финального Reduce-этапа и минимизирует нагрузку на RAM.

2. **Детерминированная сортировка:** Для гарантии воспроизводимости результатов и корректной верификации с DuckDB добавлена вторичная сортировка по алфавиту (`ORDER BY cnt DESC, combo ASC`), что устраняет расхождения из-за tie-breaker при одинаковых частотах.

3. **Безопасная сериализация:** Использование `json.dumps()` для кортежей навыков предотвращает ошибки парсинга, если название навыка содержит запятые (например, «C, C++»).

9. Итоговые выводы

1. Экспериментально подтверждена работоспособность пайплайна: данные успешно загружены в объектное хранилище RustFS, обработаны двумя независимыми методами (MapReduce и DuckDB), результаты верифицированы и совпадают бит-в-бит.

2. Строгая IT-фильтрация позволила отсеять нерелевантные вакансии и получить репрезентативную выборку мобильных разработчиков с медианной зарплатой 150 000 руб.

3. Анализ навыков выявил статистически значимые маркеры уровня позиции: профессиональные связки (kotlin + coroutines, swift + ci/cd) повышают зарплату, а общие soft skills (грамотная речь, пользователь пк) коррелируют с более низкими грейдами.

4. Региональный анализ показал высокую зрелость российского IT-рынка: разрыв между Москвой и Екатеринбургом составляет менее 20%, а базовый стек навыков идентичен во всех регионах.

5. Профилирование продемонстрировало преимущества DuckDB в скорости и эффективности памяти, но подтвердило стабильность и масштабируемость подхода MapReduce для обработки больших данных.